

This protocol will take you through the SCRMshaw pipeline from selection of a genome through to a final set of SCRMshaw predictions. You must have account privileges at CCR in order to run this protocol.

Note: Make sure to following the file naming conventions as specified in this protocol. Many of the necessary scripts will fail if the correct file names are not used. In general, italicized names such as “*my_genome*” should be replaced with the actual name of the genome with which you are working.

Part A: select and download genome

1. Select a species from the “SCRMshaw_queue” GoogleSheet
 - a. Request access for this sheet if you do not have it
 - b. URL is
https://docs.google.com/spreadsheets/d/12W3IQR49fzehndzdsAm_gqnQKuKQo2iI1JdxFv-2I4/edit?usp=sharing
 - c. Select higher priority genomes before lower priority ones
 - d. Fill out the “SCRMshander” (you!), Status, and Status_date fields
 - e. Copy the “RefSeq” ID for use in Step 3

2. Create a directory for the new genome in the “genomes” directory (within /projects/academic/mshalfon)

```
$ mkdir genomes/my_genome
```

“*my_genome*” should be the name of the species (e.g. *genus_species*)

3. Run the download script “getGenomes.sh”

```
$ cd my_genome  
$ /projects/academic/mshalfon/Scripts/getGenomes.sh GCF_XXXXXX
```

where “GCF_XXXXXX” is the RefSeq accession number for your genome.

4. If you receive a warning that the GFFv3 or protein.faa was not found, check the directory and then check the NCBI site. If these files are not available, delete your download and all associated files. Indicate on the master spreadsheet that the necessary files do not yet exist (status=”insufficient data”).

5. Otherwise, your directory should now look like this:

```
$ ls  
genome.gff  
protein.faa  
GCF_XXXXXXXXX.fna  
assembly_data_report.jsonl
```

The file “assembly_data_report.jsonl” contains metadata about the assembly and release and should be stored for future reference. You can use the NCBI “dataformat” tool to convert this to more readable formats.

6. To avoid confusion later on, change the name of the “genome.gff” and “protein.faa” files to “my_genome.gff” and “my_genome_protein.fs”.

```
$ mv genome.gff my_genome.gff
$ mv protein.faa my_genome_protein.fs
```

7. Make sure the directory and all files have group read and write permissions (check using `ls -l`) and that the group ownership is correct (grp-mshalfon). If necessary, change the group ownership (using ‘`chgrp grp-mshalfon filename`’ or ‘`chgrp -hR grp-mshalfon /directory_name`’) and/or permissions (using ‘`chmod g+rw filename`’).

Part B: Preflight

1. Run *preflight-scrmshaw* on your genome as follows:

```
$ perl /projects/academic/mshalfon/Scripts/preflight-scrmshaw.pl -gff
my_genome.gff -fasta GCF_XXXXXXXXX.fna
```

Preflight validates the formats of these files and produces a comprehensive log file that highlights any issues along with basic information such as the number of chromosomes/scaffolds and their sizes, data types present in the annotation (e.g., ‘gene’, ‘exon’, ‘ncRNA’, etc.), and average intergenic distances. *Preflight* also provides a sample output of the SCRMshaw-generated ‘gene’ and ‘exon’ files. This feature allows users to identify any discrepancies or errors stemming from the input files and to reformat these files as needed before running SCRMshaw. Be sure to check the following:

- In the first section, no GFF3 errors are reported
- Gene and Exon data appear in the following sections
- The ‘FASTA’ section reports “all sequences have proper characters”
- The following section reports that “All seqids in GFF are also in FASTA”

If any of these conditions is not met, the input files must be corrected before moving on to run SCRMshaw, or SCRMshaw will fail.

2. Remove any scaffolds not containing annotated genes. The necessary CCR modules must be loaded first:

```
$ module load foss
$ module load biopython
```

```
$ python  
/projects/academic/mshalfon/Scripts/extract_unannotatedScaffolds.py  
-po preflight_output_file.txt -f GCF_XXXXXXXX.fna
```

Part C: Repeat Mask genome

1. Run *TRF* on your “mappedOnly” genome file (e.g., mappedOnly_GCF_XXXXXXXX.fna) using the “Slurm_TRF.sh” script. (This assumes you have removed scaffolds as described above in Step B2. If you have not done this, substitute your original genome file for the “mappedOnly” file in this step.) The name of the “mappedOnly” genome file should be provided as a command line argument:

```
$ sbatch /projects/academic/mshalfon/Scripts/Slurm_TRF.sh  
mappedOnly_GCF_XXXXXXXX.fna
```

(If you want email notification when the job completes, first copy the script to your own directory and then edit the “#SBATCH --mail-user” line.)

Note: Different genomes have different repeat content and character. We are currently investigating whether the TRF parameters should be adjusted on a per-genome basis. However, we have found the parameters to be generally effective for a wide range of genomes. To change the parameters, copy the Slurm script to your own directory and edit the script.

Part D: Run SCRMshaw

1. Before running SCRMshaw on a new species, make sure you have freed up room in the scratch directory by deleting any previous SCRMshaw runs that you’ve done (see Part F).
2. Create a subdirectory for your genome in our lab’s scratch directory and navigate to this directory:

```
$ cd /vscratch/grp-mshalfon  
$ mkdir my_genome  
$ cd my_genome
```
3. Place a copy of the SCRMshaw_Slurm_template into this directory, renaming it with the name of your genome:

```
$ cp /projects/academic/mshalfon/SCRMshaw_Slurm_template.sh  
./Slurm_my_genome.sh
```

4. Edit the Slurm script as appropriate (e.g., using “*nano*”). Be sure to update at least the following lines:

```
#SBATCH --job-name → make this “my_genome_date”
#SBATCH --output → this should be “my_genome_date_slurm_log”
#SBATCH --mail-user
```

```
ARGS → (make sure the paths properly reflect the genome, annotation, and
training sets you are using. Typically, you can leave “--trainingdir1st” set to
“default_training_set.lst”)
```

The genome should be the masked genome generated in Part C.

Note: For large genomes, you may not have enough time/space available to run all of the training sets at once. In these cases, you can sequentially use the three files “default_list_1third.lst”, “default_list_2third”, and “default_list_3third.lst”.

5. Run SCRMshaw:
\$ sbatch Slurm_my_genome.sh
6. When you receive the email telling you the job is complete, check the logs to make sure SCRMshaw has run correctly.

There will be two sets of logs: a Slurm log (“my_genome_date_slurm_log”) and 25 SCRMshaw logs (“HDtestPredictions...”). The Slurm log will record the progress of the Slurm script and any errors that occur when running the script. The SCRMshaw logs will record progress and errors of SCRMshaw itself, one for each of the 25 instances run for SCRMshawHD.

The SCRMshaw results themselves will be in a set of directories titled “task_offset_m_n” (where “m” and “n” are numbers). For a description of the files within these directories, see the original SCRMshaw protocol in Kazemian and Halfon (2019) *Methods in Molecular Biology*.

Part E: Post process

1. Run the post-processing Slurm script (from within the /vscratch/grp-mshalfon/my_genome directory). The path to the GFF annotation file that was used for SCRMshaw should be entered as a command line argument using the “-i” flag. If there is no plan to run the Orthology step (Part F), then it is recommended to also include the “-g” flag, which creates GFFv3-formatted results. This option is not necessary if the Orthology step is going to be run.

```
$ sbatch
  /projects/academic/mshalfon/Scripts/Slurm_postProcessing_HalfonLab.sh
  -i /projects/academic/mshalfon/genomes/my_genome/my_genome.gff
```

(If you want email notification when the job completes, first copy the script to your own directory and then edit the “#SBATCH --mail-user” line.)

For a description of the files that will be created from this step, see the extended protocol at *protocols.io*.

Part F: Orthology

1. Create a new subdirectory in the scratch project directory. This directory should be named “Project_OM” and must be empty.

```
$ cd /vscratch/grp-mshalfon/my_genome
$ mkdir Project_OM
```

2. Navigate to the “Project_OM” directory

```
$ cd Project_OM
```

3. Run the “orthologer_slurm.sh” script from this directory. The path to protein fasta for your genome (from step A6) must go on the command line.

```
$ sbatch /projects/academic/mshalfon/Scripts/orthologer_slurm.sh
  /projects/academic/mshalfon/genomes/my_new_genome/my_new_genome_protein
  .fs
```

The results files will be created in a directory “*my_genome_orthologer_output*” in your project “*my_genome*” directory:

```
$ ls /vscratch/grp-mshalfon/my_genome/my_genome_orthologer_output
  mydata.og_map
  DMEL_protein.fs.maptxt
  my_genome_protein.fs.maptxt
```

You can check the Slurm and the Orthologer output logs to help troubleshoot any errors. These logs will be in the *my_genome* directory named “orthologer_slurm_output_log” and “*my_genome_outputlog*” respectively.

4. The next step is the only truly complicated step of the whole SCRMshaw process, and may require a bit of file reformatting and/or some coding ability. Get help if necessary! This step cannot be automated easily because the genome annotation GFF files we are

using are not all uniform from species to species. The below will work for one common version, but may need to be changed for some annotation files.

5. Return to the “*my_genome*” directory.

```
$cd ..
```

6. Generate a gene ID ⇔ protein ID mapping file. This is sample code only—see note at step F4 above.

Sample code for generating a gene ⇔ protein mapping file (Using sed):

```
$ sed -n '/#/d; /CDS/ {
s/. *gene=\([^;]*\);.*protein_id=\([^;]*\).*\1\t\2/; p; }'
/projects/academic/mshalfon/genomes/my_new_genome/my_new_genome.gff |
sort -u > my_genome.map
```

Note: Because there are often multiple CDS lines for a single gene, the above command is piped through “sort -u” to remove any duplicate lines. We recommend this step of removing duplicate lines, either at this step or following final editing of the mapping file, before running the “OM_mappingFlyOrthologsToSCRMshawPredictions.py” script.

7. Open the “*my_genome.map*” file, and make sure that Column 1 matches the gene IDs that are present in Column 6 of the “peaks_AllSets.bed” file. Also make sure that Column 2 matches the protein IDs in the “*my_genome_orthologer_output/mydata.og_map*” file. If necessary, edit the “*my_genome.map*” so that the respective columns match. (See the protocol at *protocols.io* for one example.) *The mapping steps will fail if the IDs in each of the columns do not match their counterparts in the other files.*

Make sure to record any and all manipulations in your lab notes!

8. Name the final edited file “*my_genome.finalEdited.map*”.
9. Run the “OM_mapping_slurm.sh” script

```
$ sbatch /projects/academic/mshalfon/Scripts/OM_mapping_slurm.sh
```

If you want to generate the final SCRMshaw output in GFF format, include the -g flag on the command line followed by the path to the annotation GFF (i.e. */projects/academic/mshalfon/genomes/my_new_genome/my_new_genome.gff*). This step is not generally recommended.

10. This script will produce the following files in the *my_genome* directory:

```
peaks_AllSets_my_genome.bed => the original SCRMshaw post-processed output
SO_peaks_AllSets_my_genome.bed => the SCRMshaw output following orthology
mapping
```

`S0_merged_peaks_AllSets_my_genome.bed` => the orthology-mapped SCRMshaw output following BEDTools sort+merge

If the GFF option is provided:

`S0_merged_peaks_AllSets_my_genome.gff` => the orthology-mapped SCRMshaw output in GFF format

`combined_annotation_my_genome.gff` => the orthology-mapped SCRMshaw output in GFF format appended to the genome annotation GFF file

`combined_sorted_annotation_my_genome.gff` => the orthology-mapped SCRMshaw output in GFF format appended to the genome annotation GFF file, sorted using *gff3-toolkit*

Part G: Transfer and cleanup

Files in the 'scratch' space are automatically deleted after 60 days. To properly archive your results, you must transfer the relevant data back to our lab "projects" directory.

1. Run the "transfer_SCRMshaw_results.sh" script from the project directory (i.e. `/vscratch/grp-mshalfon/my_genome`)

```
$ /projects/academic/mshalfon/Scripts/transfer_SCRMshaw_results.sh
```

This script will create a directory

`"/projects/academic/mshalfon/SCRMshaw_results/my_genome_results"` and move all the relevant files into it.

2. Retain the remaining files and directories in the scratch directory until you are sure that you no longer need them (e.g., you are not going to run additional training sets or do not need to troubleshoot your SCRMshaw run by looking at specific outputs from the initial run). Once you are satisfied of this, run the cleanup script to delete the remaining files to free up space on the scratch drive.

```
$ /projects/academic/mshalfon/Scripts/delete_SCRMshaw_projectdir.sh  
/vscratch/grp-mshalfon/my_genome
```

ONLY RUN THIS SCRIPT IF YOU HAVE COPIED ALL THE RELEVANT DATA TO THE LAB PROJECT SPACE USING THE "transfer_SCRMshaw_results.sh" SCRIPT (step G1) AND HAVE CONFIRMED SUCCESSFUL TRANSFER OF ALL RESULTS!!

**To make a slim version of the SCRMshaw project directory while not deleting the main results, you can run the script “cleanup_fastadirectory_SCRMshawHD.sh” (in the Scripts directory) instead.*

3. Delete the Slurm log file

```
$ rm my_genome_delete.log
```

4. Check that the group permissions in “/projects/academic/mshalfon/SCRMshaw_results/my_genome_results” are set to “grp-mshalfon” and have group read and write permissions (i.e., `ls -l “/projects/academic/mshalfon/SCRMshaw_results”`) and fix them if necessary (see step A7)
5. Similarly, check that permissions for the genome and annotation in the “/projects/academic/mshalfon/genomes/my_genome” directory have group rw permissions.
6. Return to the SCRMshaw queue GoogleDoc (step A1) and update the status of your genome to “Completed”.